

# Day 14 - Capstone Project: Expense Tracker

## Day 14 — Capstone Project: Expense Tracker

### Objectives

Build a complete, small application using nearly everything from the last 13 days, mostly unassisted:

- OOP (Days 8-9): classes for `Expense` and `ExpenseTracker`, dunder methods
- Generators/decorators (Day 10): a generator for filtering, a decorator for logging or timing
- File/data handling (Days 5, 11): JSON persistence, CSV import/export
- Errors (Day 6): custom exceptions, clean handling of bad input
- Testing (Day 12): a real pytest test suite, written by you, not given to you
- Structure (Day 13): proper `pyproject.toml` / src-layout / `.gitignore`

This is intentionally the least hand-held day. You get a specification (a written description of what the finished thing needs to do), not a scaffold (partially-written starter code). Expect to get stuck — that's the design, not a bug in the lesson. Thirteen days ago you didn't know what a variable was; today's exercise is deliberately close to "here's a real, if small, professional software task — go build it," because that's genuinely the point of the whole two weeks: proving to yourself that you can take a plain-English description of a program and turn it into working, tested code without someone walking you through each individual line.

### The brief

Build a CLI expense tracker, `expense_tracker`, that:

- 1 **Adds expenses:** `python -m expense_tracker add 42.50 "Groceries" --category food`
  - Each expense has: amount (float), description (str), category (str, defaults to `"general"`), and a timestamp (use `datetime.now()` when created)
- 1 **Lists expenses**, optionally filtered by category: `python -m expense_tracker list [--category food]`
- 2 **Shows a summary:** `python -m expense_tracker summary` — total spent, and a breakdown of total per category
- 3 **Exports to CSV:** `python -m expense_tracker export expenses.csv`
- 4 **Persists between runs** as JSON (same idea as Day 7, now inside a proper class rather than free functions)

### Required design elements (this is the point of the exercise — don't skip these to save time)

- An `Expense` class (Day 8-9) with a `__repr__` and, ideally, comparison support so you can sort expenses by amount or date.

- An `ExpenseTracker` class that owns a list of `Expense` objects and exposes methods like `add_expense`, `filter_by_category` (make this a **generator method**, yielding matches lazily — Day 10), `total`, `total_by_category`.
- A custom exception, e.g. `InvalidExpenseError`, raised when amount is negative or description is empty — caught at the CLI layer and shown as a clean message, never a raw traceback.
- A decorator (Day 10) applied to at least one method — e.g. logging every call to `add_expense` with a timestamp and arguments, using your own `@log_call` decorator (don't just use a library for this one — build it, to prove you understand the mechanism).
- Type hints (Day 12) on every function/method signature.
- A real `tests/` directory with pytest tests covering: adding a valid expense, adding an invalid expense (raises), filtering by category, total calculations, and CSV export producing a file with the right rows.
- Proper project structure (Day 13): `pyproject.toml`, `src/expense_tracker/`, `.gitignore`, and if you have git, real commits with meaningful messages as you go (don't do it all in one commit at the end — that defeats the point of the discipline you practiced Day 13).

## Suggested (not mandatory) file layout

```

day14-capstone/
├── pyproject.toml
├── .gitignore
├── src/
│   └── expense_tracker/
│       ├── __init__.py
│       ├── models.py           # Expense, InvalidExpenseError
│       ├── tracker.py         # ExpenseTracker, log_call decorator
│       ├── storage.py         # load/save JSON, export CSV
│       └── __main__.py        # CLI entry point (so `python -m expense_tracker` works)
└── tests/
    ├── test_models.py
    └── test_tracker.py

```

`__main__.py` is what makes `python -m expense_tracker ...` work — Python looks for that specific filename when you run a package with `-m`.

## How to approach 2+ hours of mostly-unassisted work without stalling out

- 1 **Write the `Expense` class first, in isolation, with a quick manual test in a REPL** (`python` interactive shell) before touching the CLI or storage at all. Confirm it behaves before building on top of it.
- 2 **Build `ExpenseTracker` next**, still testing manually in the REPL — add a few expenses, call `.total()`, before wiring up JSON persistence.
- 3 **Add JSON save/load**, confirm round-tripping works (this is where you'll hit the "how do I turn an `Expense` object into JSON and back" problem — you'll need to convert to/from a dict yourself; JSON doesn't know about your custom classes).
- 4 **Only then** wire up the CLI (`sys.argv` parsing like Day 7, or `argparse` if you want to explore something new — either is fine here).

- 5 **Write tests as you go**, not all at the end — write a test for `Expense` right after you write the class, not after the whole project is "done."
- 6 If you get stuck for more than ~15 minutes on one specific thing, that's a reasonable point to peek at the relevant part of `solution/` for just that piece — not the whole file.

## A hint on the JSON-with-custom-objects problem, since it's genuinely new today

You haven't been shown this pattern yet on purpose — figuring it out is part of the exercise — but here's a nudge: `json.dump/json.load` only understand the built-in types from Day 11's table. To persist an `Expense`, you need to convert it to a plain dict before dumping, and rebuild an `Expense` from a dict after loading:

```
def to_dict(self) -> dict:
    return {"amount": self.amount, "description": self.description, ...}

@classmethod
def from_dict(cls, data: dict) -> "Expense":
    return cls(data["amount"], data["description"], ...)
```

`@classmethod` is a decorator you haven't formally covered — it marks a method that receives the `*class*` (conventionally named `cls`) instead of an instance as its first argument, making it useful as an alternate constructor. Look up `@classmethod` and `@staticmethod` briefly today; both are common enough in real code that you should recognize them, even though today is the first time you're using one for real.

## Definition of done

- All 5 CLI commands work end to end, tested manually.
- `pytest` passes with no failures, and covers the required cases listed above.
- No raw tracebacks reach the user for the invalid-input cases you're asked to handle.
- Project installs with `pip install -e .` and runs via `python -m expense_tracker`.

## Reference solution

`solution/` has a complete working implementation. Give yourself a genuine attempt — ideally the full 2 hours — before opening it. The value of this whole two weeks culminates here: if you can build this mostly on your own, you're at the level this track promised.